

Diferencias entre Contenedor Web y Servidor Web

Estático vs. Dinámico

Ing. Alberto Manases Turcios Ortiz
Gestión de Servidores Web – Sección A
Lunes 20 de julio de 2026



Encuadre de la clase

Unidad 1: Creación de contenedores web para la ejecución de servlets

Horario: 1:00 pm – 2:40 pm

Duración: 1 hora 40 minutos

Objetivos de la clase

1. Distinguir con precisión qué hace un servidor web y qué hace un contenedor web.
2. Clasificar software real en una de las dos categorías (o ambas).
3. Construir en TypeScript dos versiones de "servidor": una estática y una dinámica.



Conexión con la Clase 2

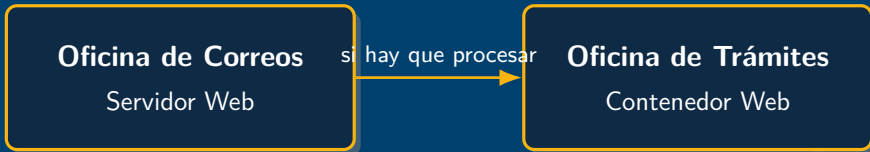
Recordatorio: en la Clase 2 construimos un mini-contenedor en TypeScript que recibía una petición, decidía qué código ejecutar (`/saludo`) y devolvía una respuesta dinámica.

Hoy: vamos a separar con total claridad **qué parte de eso era "servidor"** y **qué parte era "contenedor"** – porque en la vida real, casi nunca son la misma pieza de software.

En Clase 2 mezclamos ambos roles a propósito (un solo `http.createServer`) para ver el flujo completo. Hoy separamos esos dos roles que viven mezclados.



Correos vs. trámites



- La **oficina de correos** (servidor web) recibe paquetes y los entrega. No le importa qué hay adentro – solo los **mueve de un punto a otro** lo más rápido posible.
- La **oficina de trámites** (contenedor web) recibe una solicitud, **la procesa** (revisa documentos, hace cálculos, consulta un sistema) y produce un documento nuevo como resultado.



¿Cuál oficina necesito?

Basta la oficina de correos

"Entrégueme este sobre ya cerrado" = una imagen, un archivo CSS, un PDF. El contenido **ya existe**, tal cual, en un estante. Solo hay que ir a buscarlo y entregarlo.

Se necesita la oficina de trámites

"Calcúleme el monto de mi pensión" = un cálculo, una consulta a base de datos. El resultado **no existe todavía**: se produce en el momento, según quién pregunta y qué pregunta.

Es la misma regla práctica de la Clase 2, vista desde otro ángulo: estático vs. dinámico.



Servidor web (puro): qué hace

- Responde peticiones HTTP/HTTPS.
- Entrega **contenido estático**: el archivo ya existe en disco tal cual se entrega (HTML, CSS, imágenes, PDF).
- No "piensa": solo busca el archivo y lo entrega – sin lógica de negocio, sin sesiones.

Ejemplos de la industria

Apache HTTP Server, Nginx, IIS (en su rol más básico de servir archivos).

*Pregunta clave: si apago y vuelvo a prender el servidor, ¿el archivo entregado cambia?
Para un servidor puro, **no**.*



Contenedor web: qué hace

- Ejecuta **contenido dinámico**: el resultado **no existe todavía**, se calcula en el momento de la petición.
- Gestiona sesiones de usuario, seguridad, y la ejecución de la lógica de negocio.
- Decide, según la ruta y los datos, **qué código correr** – exactamente lo que programamos a mano en la Clase 2.

Ejemplos de la industria

Otros ejemplos: PHP-FPM (PHP), Unicorn (Python), Puma (Ruby). **En este curso, nuestro contenedor de libre distribución es Node.js + Express, programado en TypeScript.**



Software que combina ambos roles

Producto	Rol(es)
Node.js + Express (lo que usaremos)	Puede servir archivos estáticos y ejecutar lógica dinámica – ambos roles en uno
IIS	Servidor web + contenedor de aplicaciones (vía módulos como iisnode para Node/TypeScript)
Apache + mod_php	Servidor web + ejecuta lógica PHP embebida (clásico LAMP)
Apache HTTP Server / Nginx	Servidor web puro; en arquitecturas reales suele ir <i>delante</i> de un contenedor, sirviendo estáticos y reenviando lo dinámico



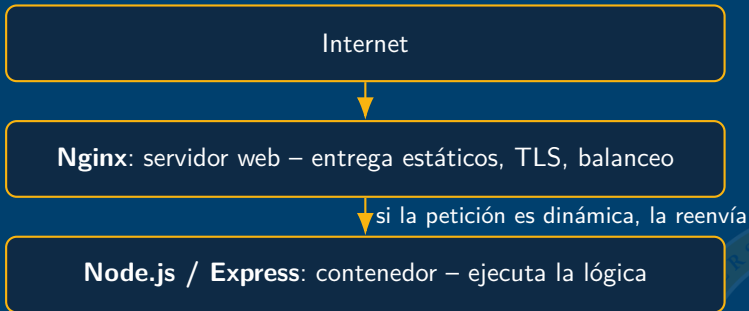
Tabla comparativa

Característica	Servidor web	Contenedor web
Qué entrega	Archivo ya existente	Resultado calculado en el momento
¿Ejecuta lógica?	No (directamente)	Sí
¿Maneja sesiones de usuario?	No	Sí
Ejemplos	Apache, Nginx	Node.js/Express, Unicorn

Esta tabla es el resumen de toda la teoría de hoy: cada fila es una pregunta que pueden usar para clasificar cualquier software que encuentren.



Arquitectura típica en producción



Esto se hace porque el servidor web puro es **más eficiente y liviano** sirviendo archivos estáticos y manejando miles de conexiones simultáneas, mientras el contenedor se concentra en la lógica.

Caso de uso real: así está armado, por ejemplo, un sitio de e-commerce – Nginx entrega imágenes de productos y CSS; Node.js calcula el carrito y el total con impuestos.



Versión A – "Servidor" puro (solo entrega un archivo)

```
1 // servidor-estatico.ts
2 import * as http from "http";
3 import * as fs from "fs";
4 import * as path from "path";
5 const servidor = http.createServer((req, res) => {
6   const archivo = path.join(__dirname, "publico", "saludo.html");
7   fs.readFile(archivo, (error, contenido) => {
8     if (error) {
9       res.writeHead(404, { "Content-Type": "text/plain" });
10      res.end("Archivo no encontrado");
11      return;
12    }
13    // No se calcula nada. Solo se lee y se entrega tal cual.
14    res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
15    res.end(contenido);
16  });
17 });
18 servidor.listen(3001, () => console.log("Servidor estatico en :3001"));
```



Versión A: línea por línea

- `fs.readFile(...)` abre un archivo **que ya existe** en disco – no genera nada nuevo.
- El callback (`error, contenido`) recibe el contenido **tal cual está guardado**, sin tocarlo.
- No hay `if` sobre datos de usuario, no hay cálculo, no hay fecha ni nombre: la respuesta es **idéntica para cualquiera** que visite esta ruta.

Crear `publico/saludo.html` con cualquier texto fijo. Cada vez que se visita, el archivo es exactamente el mismo, **sin importar quién pregunte**.



Versión B – "Contenedor" (calcula según quién pregunta)

```
1 // contenedor-dinamico.ts
2 import * as http from "http";
3 import { URL } from "url";
4 const servidor = http.createServer((req, res) => {
5   const url = new URL(req.url ?? "/", `http://${req.headers.host}`);
6   if (url.pathname === "/hora-saludo") {
7     const nombre = url.searchParams.get("nombre") ?? "visitante";
8     const hora = new Date().getHours();
9     const saludo = hora < 12 ? "Buenos dias"
10      : hora < 19 ? "Buenas tardes" : "Buenas noches";
11     // Esto SI se calcula en cada peticion: cambia segun hora y nombre.
12     res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
13     res.end(`<h1>${saludo}, ${nombre}</h1>`);
14     return;
15   }
16   res.writeHead(404).end("No encontrado");
17 });
18 servidor.listen(3002, () => console.log("Contenedor dinamico en :3002"));
```



Versión B: línea por línea

- `new Date().getHours()` lee la hora **en el instante exacto** de la petición – nunca es un valor guardado de antemano.
- El operador ternario encadenado (`hora < 12 ? ... : hora < 19 ? ... : ...`) es la **lógica de negocio**: decide el saludo según una condición.
- nombre viene de `searchParams`: **cambia según quién pregunta**.
- Resultado: la misma ruta `/hora-saludo` responde **distinto en la mañana, en la tarde y para Ana vs. Carlos**.

Visitar `/hora-saludo?nombre=Carlos` en distintos momentos del día – la respuesta cambia, porque se calcula al momento. **Eso es, en esencia, lo que distingue a un contenedor.**



Ejecutarlas y compararlas

Correr ambas a la vez (puertos distintos, no chocan):

```
npx ts-node servidor-estatico.ts  
npx ts-node contenedor-dinamico.ts
```

Probar estático

<http://localhost:3001>

Recargar varias veces: **siempre igual**.

Probar dinámico

<http://localhost:3002/hora-saludo?nombre=Ana>

Cambiar nombre: **respuesta distinta**.



Ejercicio individual

Ejecutar ambas versiones (en puertos distintos) y clasificar 5 situaciones reales como **"servidor estático"** o **"contenedor dinámico"**:

1. Descargar el logo de una empresa.
2. Ver el saldo de tu cuenta bancaria en línea.
3. Abrir un PDF de un syllabus.
4. Ver tu feed personalizado de redes sociales.
5. Cargar el ícono de favoritos (*favicon*) de un sitio.

Pista

Para cada caso pregúntense: ¿el resultado es igual para todos, sin importar quién pregunta? Si sí, es estático. Si depende de la persona, la hora o un dato guardado, es dinámico.



Verificación de aprendizaje

1. ¿Cuál es la diferencia central entre estático y dinámico?





Verificación de aprendizaje

1. ¿Cuál es la diferencia central entre estático y dinámico?

Estático: el archivo ya existe y es siempre igual. Dinámico: se calcula en el momento y puede cambiar según quién pregunta.



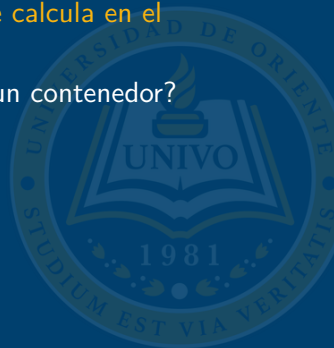


Verificación de aprendizaje

1. ¿Cuál es la diferencia central entre estático y dinámico?

Estático: el archivo ya existe y es siempre igual. Dinámico: se calcula en el momento y puede cambiar según quién pregunta.

2. ¿Por qué en producción se pone un servidor web delante de un contenedor?





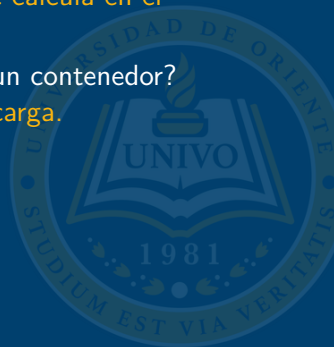
Verificación de aprendizaje

1. ¿Cuál es la diferencia central entre estático y dinámico?

Estático: el archivo ya existe y es siempre igual. Dinámico: se calcula en el momento y puede cambiar según quién pregunta.

2. ¿Por qué en producción se pone un servidor web delante de un contenedor?

Eficiencia sirviendo archivos, TLS centralizado, balanceo de carga.





Verificación de aprendizaje

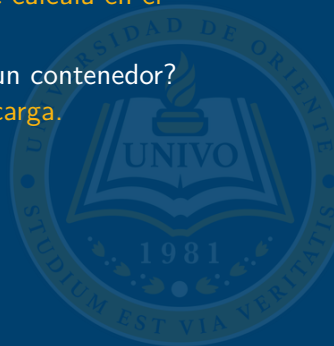
1. ¿Cuál es la diferencia central entre estático y dinámico?

Estático: el archivo ya existe y es siempre igual. Dinámico: se calcula en el momento y puede cambiar según quién pregunta.

2. ¿Por qué en producción se pone un servidor web delante de un contenedor?

Eficiencia sirviendo archivos, TLS centralizado, balanceo de carga.

3. Da un ejemplo de arquitectura combinada.





Verificación de aprendizaje

1. ¿Cuál es la diferencia central entre estático y dinámico?

Estático: el archivo ya existe y es siempre igual. Dinámico: se calcula en el momento y puede cambiar según quién pregunta.

2. ¿Por qué en producción se pone un servidor web delante de un contenedor?

Eficiencia sirviendo archivos, TLS centralizado, balanceo de carga.

3. Da un ejemplo de arquitectura combinada.

Nginx (servidor web) + Node.js/Express (contenedor).





Resumen de la clase

Rol	Función principal	Ejemplos
Servidor web	Entrega archivos, TLS, proxy	Apache, Nginx, IIS
Contenedor web	Ejecuta lógica dinámica	Node.js/Express, Unicorn
Combinado	Ambos roles	Nginx + Node.js/Express



Tarea para casa

1. Resolver el ejercicio de clasificación de las 5 situaciones reales.
2. Ejecutar `servidor-estatico.ts` y `contenedor-dinamico.ts` y documentar (máx. 1 página) qué diferencia observaron al recargar cada uno varias veces.

Para profundizar (documentación oficial)

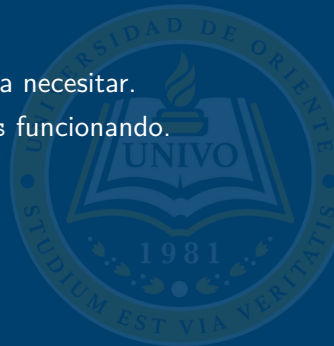
- nodejs.org – módulo `fs`
- nginx.org/docs – documentación oficial de Nginx



Preparación para la Clase 4

Tema: Contenedores web propietarios vs. de libre distribución – comparativa de licenciamiento, soporte y casos de uso.

- Repasar la tabla comparativa servidor vs. contenedor: la van a necesitar.
- Traer `servidor-estatico.ts` y `contenedor-dinamico.ts` funcionando.





Comisión de Acreditación
de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

MUCHAS GRACIAS

Te espero la próxima clase

